

Oefeningen

Documentatie

[MDN Web Docs - for](#) bevat documentatie over de `for`-loop. [MDN Web Docs - while](#) bevat documentatie over de `while`-loop. [MDN Web Docs - do...while](#) bevat documentatie over de `do while`-loop

Omschrijf de code

Omschrijf onderstaande code met de juiste termen. Gebruik de termen die in de theorie zijn besproken.

Voorbeelden:

```
for (let i = 0; i < 10; i++) {  
  console.log(i);  
}  
/**  
 * iteratieve statement – for keyword  
 * i is een variabele die wordt geïnitialiseerd met de  
waarde 0  
 * i < 10 is de voorwaarde die wordt geëvalueerd  
 * i++ hoogt de waarde van i met 1 op na elke iteratie  
 * Hetgeen tussen { en } staat is het codeblok dat elke  
iteratie wordt uitgevoerd.
```

js

* In het codeblok wordt de waarde van i gelogd naar de console.

*

* De iteratie zal 10 keer uitgevoerd worden.

*/

js

```
let i = 0;
while (i < 10) {
  console.log(i);
  i++;
}
```

/**

* Er wordt een variabele i geïnitieerd met de waarde 0.

* iteratieve statement – while keyword

* i < 10 is de voorwaarde die wordt geëvalueerd

* Het codeblok tussen { en } wordt uitgevoerd zolang de voorwaarde waar (true) is.

* In het codeblok wordt de waarde van i gelogd naar de console.

* i++ hoogt de waarde van i met 1 op voor er opnieuw wordt geëvalueerd.

*

* De iteratie zal 10 keer uitgevoerd worden.

*/

js

```
let invoer = '';
while (invoer !== 'stop') {
  invoer = prompt('Voer een waarde in (type "stop" om te stoppen):');
  if (invoer === 'stop') {
    console.log('Programma gestopt.');
```

```
/**
 * Er wordt een variabele genaamd 'invoer'
geïnitialiseerd met een lege string.
 * iteratieve statement – while keyword
 * (invoer !== 'stop') is de voorwaarde die wordt
geëvalueerd
 * Zolang de waarde van invoer niet gelijk is aan 'stop',
wordt het codeblok tussen { en } uitgevoerd.
 * In het codeblok wordt de gebruiker gevraagd om een
waarde in te voeren via prompt().
 * Als de gebruiker 'stop' invoert, evalueert de if-
statement naar true en wordt het bericht 'Programma
gestopt.' gelogd naar de console.
 * Als de gebruiker iets anders invoert, wordt het
bericht 'Je hebt ingevoerd: ' gevolgd door de waarde van
invoer gelogd naar de console.
 *
 * We weten niet hoeveel iteraties er zullen zijn, omdat
het afhankelijk is van de invoer van de gebruiker.
 *
```

Oefening 1

Via `prompt()` (dit is een bestaande `functie` in JavaScript) kan je een waarde laten invoeren door de gebruiker, dit zorgt ervoor dat er geen hard-coded waarde is, maar dat de gebruiker elke keer een andere waarde kan invoeren.

```
const score = prompt("Voer je score in:");
while (score < 10) {
  console.log("Je bent niet geslaagd.");
  score = prompt("Voer je score opnieuw in:");
}
console.log("Je bent geslaagd!");
```

js

▼ Oplossing

js

```
const score = prompt("Voer je score in:");  
  
/**  
 * 'const' is een keyword  
 * 'score' is een constante variabele  
 * = is de toekenningoperator  
 * prompt() is een functie die een invoervenster toont aan de  
gebruiker.  
 * De waarde die de gebruiker invoert wordt toegewezen aan de  
constante variabele 'score'.  
 * * De waarde van 'score' is een string, omdat prompt()  
altijd een string teruggeeft.  
 */  
  
/**  
 * while is een iteratieve statement  
 * (score < 10) is de voorwaarde die wordt geëvalueerd  
 * < is de vergelijkingsoperator  
 * score bevat de waarde die de gebruiker heeft ingevoerd  
(string)  
 * 10 is een number  
 *  
 * De voorwaarde evalueert of de waarde van 'score' kleiner  
is dan 10.  
 * Omdat 'score' een string is, wordt deze automatisch  
geconverteerd naar een number voor de vergelijking.  
 * Als de voorwaarde waar (true) is, wordt het codeblok  
tussen { en } uitgevoerd.  
 * In het codeblok wordt een bericht gelogd naar de console  
dat de gebruiker niet geslaagd is.  
 * Vervolgens wordt de gebruiker gevraagd om een nieuwe score  
in te voeren via prompt().  
 * Dit proces herhaalt zich totdat de gebruiker een score  
invoert die groter dan of gelijk aan 10 is.  
 * Zodra de gebruiker een score invoert die groter dan of  
gelijk aan 10 is, wordt de while-loop beëindigd en wordt het  
bericht "Je bent geslaagd!" gelogd naar de console.  
 */  
while (score < 10) {  
    console.log("Je bent niet geslaagd.");  
    score = prompt("Voer je score opnieuw in:");  
}
```

```
}  
console.log("Je bent geslaagd!");
```

Oefening 2

```
for (let i = 5; i > 0; i--) {  
  console.log("We tellen af, het nummer is nu: " + i);  
}
```

js

▼ Oplossing

```
for (let i = 5; i > 0; i--) {  
  console.log("We tellen af, het nummer is nu: " + i);  
}
```

/**
 * for is een iteratieve statement
 * let i = 5; is de initialisatie van de variabele i met de
waarde 5
 * i > 0 is de voorwaarde die wordt geëvalueerd
 * i-- verlaagt de waarde van i met 1 na elke iteratie
 * Het codeblok tussen { en } wordt uitgevoerd zolang de
voorwaarde waar (true) is.
 * In het codeblok wordt een bericht gelogd naar de console
met de huidige waarde van i.
 *
 * De iteratie zal 5 keer uitgevoerd worden, waarbij i
afneemt van 5 naar 1.
 */

js

Schrijf de code

for-loop

Schrijf een iteratieve statement die 3x wordt uitgevoerd, gebruik hiervoor een for-statement.

Verwachte output in de console:

```
Iteratie 1
Iteratie 2
Iteratie 3
```

sh

▼ Hint

De syntax van een `for`-statement is als volgt:

```
for (initialisatie; voorwaarde; iteratie) {
  // code die wordt uitgevoerd
}
```

js

▼ Hint

Je kan de teller gebruiken om elke iteratie een andere zin te loggen.

```
for (let teller = 1; teller < 4; teller++) {
  console.log(teller);
}

/**
 * Output:
 * 1
 * 2
 * 3
 */
```

js

Of indien je liever consistent vanaf 0 begint:

js

```
for (let teller = 0; teller < 3; teller++) {  
  console.log(teller + 1); // Omdat we willen beginnen met 1  
  in de output en we vertrekken vanaf 0, tellen we 1 op bij de  
  huidige teller wanneer we loggen.  
}  
  
/**  
 * Output:  
 * 1  
 * 2  
 * 3  
 */
```

▼ Oplossing

js

```
for (let i = 0; i < 3; i++) {  
  const getal = i + 1;  
  console.log("Iteratie " + getal);  
}
```

of

js

```
for (let i = 1; i < 4; i++) {  
  console.log("Iteratie " + i);  
}
```

of

js

```
for (let i = 1; i <= 3; i++) {  
  console.log("Iteratie " + i);  
}
```

For-loop omzetten naar een while-loop

In JavaScript is er een manier om een lijst aan gegevens te maken, dit wordt een `array` genoemd. Dit wordt in een ander hoofdstuk behandeld, maar om deze oefening te maken is het niet nodig om de details van arrays te kennen.

Wat belangrijk is om te weten is dat je een array kan maken met de volgende syntax:

```
const array = [waarde1, waarde2, waarde3];
```

js

Concreet voorbeeld:

```
const kleuren = ["rood", "groen", "blauw"];
```

js

Verder is het belangrijk om te weten dat we een item in een array kunnen opvragen aan de hand van de index. De index begint bij 0 en gaat tot de lengte van de array min 1. De lengte van de array `kleuren` is 3, dus de indexen zijn 0, 1 en 2.

```
const kleuren = ["rood", "groen", "blauw"];
```

js

```
// Index 0: "rood"
```

```
// Index 1: "groen"
```

```
// Index 2: "blauw"
```

```
// Om het eerste item in de array op te vragen, gebruik  
je:
```

```
console.log(kleuren[0]); // Output: "rood"
```

```
// Om het tweede item in de array op te vragen, gebruik  
je:
```

```
console.log(kleuren[1]); // Output: "groen"
```

```
// Om het derde item in de array op te vragen, gebruik
```



```
je:
console.log(kleuren[2]); // Output: "blauw"

// Om de lengte van een array op te vragen, gebruik je:
console.log(kleuren.length); // Output: 3
```

Gebruikmakend van deze kennis, bestudeer onderstaande code en beschrijf wat er gebeurt.

```
const kleuren = ["rood", "groen", "blauw"];
for (let i = 0; i < kleuren.length; i++) {
  console.log("De kleur is: " + kleuren[i]);
}
```

js

▼ Oplossing

```
const kleuren = ["rood", "groen", "blauw"];
/**
 * Er wordt een constante variabele 'kleuren' gedefinieerd
 * waaraan een array wordt toegekend met drie kleuren (strings).
 */

for (let i = 0; i < kleuren.length; i++) {
  console.log("De kleur is: " + kleuren[i]);
}
/*
 * for is een iteratieve statement
 * let i = 0; is de initialisatie van de variabele i met de
 * waarde
 * i < kleuren.length is de voorwaarde die wordt geëvalueerd
 * kleuren.length geeft de lengte van de array 'kleuren', wat
 * 3 is in dit geval.
 * i++ verhoogt de waarde van i met 1 na elke iteratie
 * Het codeblok tussen { en } wordt uitgevoerd zolang de
 * voorwaarde waar (true) is.
 * In het codeblok wordt een bericht gelogd naar de console
 * met de huidige kleur uit de array 'kleuren'.
 * De teller i wordt gebruikt als index om de kleur uit de
 * array op te vragen.
```

js

```
* De iteratie zal 3 keer uitgevoerd worden, waarbij i de
waarden 0, 1 en 2 aanneemt.
*
* De eerste iteratie staat er eigenlijk "De kleur is: " +
kleuren[0], wat "De kleur is: rood" zal loggen.
* De tweede iteratie staat er "De kleur is: " + kleuren[1],
wat "De kleur is: groen" zal loggen.
* De derde iteratie staat er "De kleur is: " + kleuren[2],
wat "De kleur is: blauw" zal loggen
*/
```

Gebruik bovenstaande kennis om de for-loop naar een while-loop te herschrijven.

▼ Oplossing

```
const kleuren = ["rood", "groen", "blauw"];

let i = 0; // Initialiseer de teller i met 0
while (i < kleuren.length) {
  console.log("De kleur is: " + kleuren[i]);
  i++; // Verhoog de teller i met 1 voor de volgende
  iteratie
}
```

js

Som van getallen

Schrijf een iteratieve statement die de getallen van 1 tot en met 10 optelt en het resultaat in de console logt.

Verwachte output in de console:

```
De som van de getallen van 1 tot en met 10 is: 55
```

sh

▼ Hint

Gebruik een `for`-statement om de iteratie uit te voeren. Je kan een variabele declareren voor het definiëren van de for-loop waarin het tussentijds resultaat bijgehouden kan worden.

▼ Oplossing

```
js
let som = 0; // Variabele om de som bij te houden

for (let i = 1; i <= 10; i++) {
  som = som + i; // Voeg de huidige waarde van i toe aan de som
}

console.log("De som van de getallen van 1 tot en met 10 is: " + som);
```

Bestaande code uitbreiden (uitdaging)

Je wordt gevraagd om extra functionaliteit toe te voegen aan bestaande code.

```
js
let invoer = prompt("Speler 1, blad - steen - schaar?");
let invoer2 = prompt("Speler 2, blad - steen - schaar?");

if (invoer === invoer2) {
  alert("Gelijkspel!");
} else if (
  (invoer === "blad" && invoer2 === "steen") ||
  (invoer === "steen" && invoer2 === "schaar") ||
  (invoer === "schaar" && invoer2 === "blad")
) {
  alert("Speler 1 wint!");
} else if (
  (invoer2 === "blad" && invoer === "steen") ||
  (invoer2 === "steen" && invoer === "schaar") ||
```

```
(invoer2 === "schaar" && invoer === "blad")
) {
  alert("Speler 2 wint!");
}
```

Je moet de code van het `blad-steen-schaar`-spel aanpassen zodat de gebruiker kan blijven spelen totdat 1 van de spelers `STOP` invoert. Als een van de spelers `STOP` invoert, moet het spel eindigen en moet er een bericht worden weergegeven dat het spel is gestopt. Het is oké dat speler 2 nog steeds moet invoeren, zelfs al heeft speler 1 `STOP` ingevoerd.

▼ Hint

Je kan een `while` - of `do while` -loop gebruiken om de code te herhalen totdat een van de spelers `STOP` invoert. Zorg ervoor dat je de invoer van beide spelers controleert in elke iteratie van de loop.

▼ Hint

Gebruik de `||` -operator om meerdere voorwaarden te combineren. De `||` -operator staat voor "of" en evalueert naar `true` als een van de voorwaarden waar is.

Gebruik de `&&` -operator om meerdere voorwaarden te combineren. De `&&` -operator staat voor "en" en evalueert naar `true` als beide voorwaarden waar zijn.

▼ Oplossing

```
let invoer;
let invoer2;

while (invoer !== "STOP" && invoer2 !== "STOP") {
  invoer = prompt("Speler 1, blad - steen - schaar? (of STOP om te stoppen)");
  invoer2 = prompt("Speler 2, blad - steen - schaar? (of STOP
```

js

```

om te stoppen));

    if (invoer === "STOP" || invoer2 === "STOP") {
        alert("Het spel is gestopt.");
    } else if (invoer === invoer2) {
        alert("Gelijkspel!");
    } else if (
        (invoer === "blad" && invoer2 === "steen") ||
        (invoer === "steen" && invoer2 === "schaar") ||
        (invoer === "schaar" && invoer2 === "blad")
    ) {
        alert("Speler 1 wint!");
    } else if (
        (invoer2 === "blad" && invoer === "steen") ||
        (invoer2 === "steen" && invoer === "schaar") ||
        (invoer2 === "schaar" && invoer === "blad")
    ) {
        alert("Speler 2 wint!");
    } else {
        alert("Ongeldige invoer!");
    }
}

```

Het is mogelijk om gebruik te maken van `break` om de loop vervroegd te beëindigen, i.p.v. alles in een `if else`-structuur te stoppen. Dit maakt de code iets leesbaarder.

```

let invoer;
let invoer2;

while (invoer !== "STOP" && invoer2 !== "STOP") {
    invoer = prompt("Speler 1, blad - steen - schaar? (of STOP om te stoppen)");
    invoer2 = prompt("Speler 2, blad - steen - schaar? (of STOP om te stoppen)");

    if (invoer === "STOP" || invoer2 === "STOP") {
        alert("Het spel is gestopt.");
        break;
    }

    if (invoer === invoer2) {
        alert("Gelijkspel!");
    }
}

```

js

```

    } else if (
      (invoer === "blad" && invoer2 === "steen") ||
      (invoer === "steen" && invoer2 === "schaar") ||
      (invoer === "schaar" && invoer2 === "blad")
    ) {
      alert("Speler 1 wint!");
    } else if (
      (invoer2 === "blad" && invoer === "steen") ||
      (invoer2 === "steen" && invoer === "schaar") ||
      (invoer2 === "schaar" && invoer === "blad")
    ) {
      alert("Speler 2 wint!");
    } else {
      alert("Ongeldige invoer!");
    }
  }
}

```

Voor de mensen die de `do while` -loop willen gebruiken, kan dit ook:

```

let invoer;
let invoer2;

do {
  invoer = prompt("Speler 1, blad - steen - schaar? (of STOP om te stoppen)");
  invoer2 = prompt("Speler 2, blad - steen - schaar? (of STOP om te stoppen)");

  if (invoer === "STOP" || invoer2 === "STOP") {
    alert("Het spel is gestopt.");
    break;
  }

  if (invoer === invoer2) {
    alert("Gelijkspel!");
  } else if (
    (invoer === "blad" && invoer2 === "steen") ||
    (invoer === "steen" && invoer2 === "schaar") ||
    (invoer === "schaar" && invoer2 === "blad")
  ) {
    alert("Speler 1 wint!");
  } else if (
    (invoer2 === "blad" && invoer === "steen") ||

```

js

```

    (invoer2 === "steen" && invoer === "schaar") ||
    (invoer2 === "schaar" && invoer === "blad")
  ) {
    alert("Speler 2 wint!");
  } else {
    alert("Ongeldige invoer!");
  }
} while (invoer !== "STOP" && invoer2 !== "STOP");

```

Het verschil? Initialiseer `invoer` of `invoer2` met `STOP` en voer beide oplossingen uit. Je zal zien dat de `while` -loop nooit wordt uitgevoerd omdat de voorwaarde `false` is waardoor het codeblok niet wordt uitgevoerd. Bij de `do while` -loop wordt het codeblok minstens één keer uitgevoerd en daarna pas wordt de voorwaarde geëvalueerd. Er wordt dus altijd minstens één keer gevraagd om invoer uit te voeren, zelfs als de initiële waarde `STOP` is.

Extra uitdaging: zoek op hoe je kan zorgen dat de invoer altijd in kleine letters wordt omgezet, zodat zowel `SCHAAR`, `Schaar` als `schaar` als `schaar` worden geïnterpreteerd. Dit zorgt voor een betere gebruikerservaring, omdat de gebruiker niet hoeft te letten op hoofdletters of kleine letters.

▼ Hint

Je kan de `toLowerCase()` -methode gebruiken om een string om te zetten naar kleine letters. [MDN Web Docs](#) bevat documentatie over deze methode.

▼ Oplossing

```

let invoer;
let invoer2;

while (invoer !== "STOP" && invoer2 !== "STOP") {
  invoer = prompt("Speler 1, blad - steen - schaar? (of STOP om te stoppen)");
  invoer2 = prompt("Speler 2, blad - steen - schaar? (of STOP om te stoppen)");
}

```

js

```

invoer = invoer.toLowerCase();
invoer2 = invoer2.toLowerCase();

if (invoer === "STOP" || invoer2 === "STOP") {
    alert("Het spel is gestopt.");
    break;
}

if (invoer === invoer2) {
    alert("Gelijkspel!");
} else if (
    (invoer === "blad" && invoer2 === "steen") ||
    (invoer === "steen" && invoer2 === "schaar") ||
    (invoer === "schaar" && invoer2 === "blad")
) {
    alert("Speler 1 wint!");
} else if (
    (invoer2 === "blad" && invoer === "steen") ||
    (invoer2 === "steen" && invoer === "schaar") ||
    (invoer2 === "schaar" && invoer === "blad")
) {
    alert("Speler 2 wint!");
} else {
    alert("Ongeldige invoer!");
}
}

```

Na het toevoegen van de extra functionaliteit is er een bug in de code geslopen, als de speler op **Annuleren** klikt i.p.v. een waarde in te voeren, dan wordt er **null** toegekend aan **invoer** of **invoer2**. Dit zorgt voor een foutmelding in de console omdat **.toLowerCase()** niet kan worden aangeroepen op **null**. Voeg code toe om een alert te tonen met de boodschap dat een waarde invoeren verplicht is en de loop te stoppen als de gebruiker op **Annuleren** klikt.

▼ Oplossing

```

let invoer;
let invoer2;

```

js


```

while (invoer !== "STOP" && invoer2 !== "STOP") {
    invoer = prompt("Speler 1, blad - steen - schaar? (of STOP om te stoppen)");
    invoer2 = prompt("Speler 2, blad - steen - schaar? (of STOP om te stoppen)");

    if (invoer === null || invoer2 === null) {
        alert("Het is verplicht om een waarde in te voeren");
        break;
    }

    invoer = invoer.toLowerCase();
    invoer2 = invoer2.toLowerCase();

    if (invoer === "STOP" || invoer2 === "STOP") {
        alert("Het spel is gestopt.");
        break;
    }

    if (invoer === invoer2) {
        alert("Gelijkspel!");
    } else if (
        (invoer === "blad" && invoer2 === "steen") ||
        (invoer === "steen" && invoer2 === "schaar") ||
        (invoer === "schaar" && invoer2 === "blad")
    ) {
        alert("Speler 1 wint!");
    } else if (
        (invoer2 === "blad" && invoer === "steen") ||
        (invoer2 === "steen" && invoer === "schaar") ||
        (invoer2 === "schaar" && invoer === "blad")
    ) {
        alert("Speler 2 wint!");
    } else {
        alert("Ongeldige invoer!");
    }
}

```

Je merkt op dat de code steeds langer en complexer wordt naargelang we extra functionaliteit moeten toevoegen. In een ander hoofdstuk zal je leren hoe je lange code kan opdelen in kleinere, herbruikbare stukken

code (`functies`), wat de leesbaarheid en onderhoudbaarheid van je code zal verbeteren.

Iteratieve statements met objecten en arrays

Voor onderstaande oefeningen is het belangrijk dat je weet wat een `object` en een `array` is. Je kan dit terugvinden in het hoofdstuk over `Objecten en arrays`.

Object uitlezen

Maak een object `student` met minstens drie eigenschappen (bv. `naam`, `leeftijd`, `isIngeschreven`). Gebruik een `for`-loop of een `for...in`-loop om alle eigenschappen en hun waarde in de console te loggen.

▼ Oplossing

Met `for...in`:

```
const student = {  
  naam: "John Duck",  
  leeftijd: 22,  
  isIngeschreven: true,  
};  
  
for (let key in student) {  
  console.log(`${key}: ${student[key]}`);  
}
```

js

Met `for`-loop:

```
const student = {  
  naam: "John Duck",  
  leeftijd: 22,  
}
```

js

```
    isIngeschreven: true,  
  };  
  
  const keys = Object.keys(student);  
  for (let i = 0; i < keys.length; i++) {  
    const key = keys[i];  
    console.log(`${key}: ${student[key]}`);  
  }  
}
```

Object in HTML weergeven

Gegeven is onderstaande code waarbij er een object wordt geïnitieerd in het bestand `object-in-html.js`:

```
const wagen = {  
  merk: "Opel",  
  model: "Manta 400",  
  bouwjaar: 1981,  
};
```

js

Gegeven is onderstaande CSS in het bestand `object-in-html.css`:

```
table {  
  border-collapse: collapse;  
  border: 1px solid black;  
}  
  
th,  
td {  
  border: 1px solid black;  
  padding: 0.5rem;  
  text-align: left;  
}
```

CSS

Gegeven is onderstaande HTML:

```
html

<!doctype html>
<html lang="nl">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Document</title>

    <link rel="stylesheet" href="./object-in-html.css" />
    <script type="module" src="./object-in-html.js">
</script>
  </head>
  <body>
    <table>
      <thead>
        <tr>
          <th>Eigenschap</th>
          <th>Waarde</th>
        </tr>
      </thead>
      <tbody id="wagen-info"></tbody>
    </table>
  </body>
</html>
```

Zorg ervoor dat via JavaScript de eigenschappen en waarden van het object `wagen` in de tabel worden weergegeven. Wanneer er bijvoorbeeld een extra eigenschap aan het object wordt toegevoegd, moet deze ook automatisch in de tabel verschijnen.

TIP

Bouw eerst de HTML-structuur manueel op, zo krijg je een idee van wat het eindresultaat moet zijn. Vervang daarna de manuele HTML door JavaScript die de HTML opbouwt.

▼ Oplossing

html

```
<table>
  <thead>
    <tr>
      <th>Eigenschap</th>
      <th>Waarde</th>
    </tr>
  </thead>
  <tbody id="wagen-info"></tbody>
</table>
```

CSS

```
table {
  border-collapse: collapse;
  border: 1px solid black;
}

th,
td {
  border: 1px solid black;
  padding: 0.5rem;
  text-align: left;
}
```

js

```
const wagen = {
  merk: "Opel",
  model: "Manta 400",
  bouwjaar: 1981,
};

const tbody = document.querySelector("tbody");
for (const key in wagen) {
  const value = wagen[key];

  const tr = document.createElement("tr"); // Maak een nieuwe
```

```

rij aan.
    const tdKey = document.createElement("td"); // Maak een
    nieuwe cel aan voor de key.
    tdKey.innerText = key;
    const tdValue = document.createElement("td"); // Maak een
    nieuwe cel aan voor de value.
    tdValue.innerText = value;

    tr.appendChild(tdKey); // Voeg de cel met de key toe aan de
    rij.
    tr.appendChild(tdValue); // Voeg de cel met de value toe
    aan de rij.
    tbody.appendChild(tr); // Voeg de rij toe aan de tabel
    body.
}

```

Eigenschap	Waarde
merk	Opel
model	Manta 400
bouwjaar	1981

Array uitlezen

Gegeven is onderstaande code waarbij er een array wordt geïnitieerd:

```
const dieren = ["Hond", "Kat", "Papegaai", "Hamster"];
```

js

Gebruik een `for`-loop of een `for...of`-loop om alle dieren in de console te loggen.

▼ Oplossing

Met `for...of` :

```
const dieren = ["Hond", "Kat", "Papegaai", "Hamster"];

for (let dier of dieren) {
  console.log(dier);
}
```

js

Met `for`-loop:

```
const dieren = ["Hond", "Kat", "Papegaai", "Hamster"];

for (let i = 0; i < dieren.length; i++) {
  const dier = dieren[i];
  console.log(dier);
}
```

js

Array in HTML weergeven

Gegeven is onderstaande code waarbij er een array wordt geïnitieerd in het bestand `array-in-html.js`:

```
const dieren = ["Hond", "Kat", "Papegaai", "Hamster"];
```

js

Gegeven is onderstaande CSS in het bestand `array-in-html.css`:

```
ul {
  list-style-type: square;
  padding-left: 1rem;
}
```

CSS

Gegeven is onderstaande HTML:

```
<!doctype html>
<html lang="nl">
```

html

```

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>Document</title>

  <link rel="stylesheet" href="./array-in-html.css" />
  <script defer src="./array-in-html.js"></script>
</head>
<body>
  <ul id="dierenlijst"></ul>
</body>
</html>

```

Zorg ervoor dat via JavaScript alle dieren in de array in de ongeordende lijst (`<ul>`) worden weergegeven. Wanneer er bijvoorbeeld een extra dier aan de array wordt toegevoegd, moet deze ook automatisch in de lijst verschijnen.

▼ Oplossing

```
<ul id="dierenlijst"></ul>
```

html

```

ul {
  list-style-type: square;
  padding-left: 1rem;
}

```

CSS

```

const dieren = ["Hond", "Kat", "Papegaai", "Hamster"];

const lijst = document.querySelector("#dierenlijst");

for (const dier of dieren) {
  const li = document.createElement("li");
  li.innerText = dier;
  lijst.appendChild(li);
}

```

js


```
}
```

- Hond
- Kat
- Papegaai
- Hamster

Combinatie: Objecten in Arrays

Gegeven is onderstaande code waarbij er een array van objecten wordt geïnitialiseerd in het bestand `combinatie.js`:

```
const studenten = [  
  { naam: "John Duck", leeftijd: 34, isIngeschreven: true  
},  
  { naam: "Jane Duck", leeftijd: 20, isIngeschreven:  
false },  
  { naam: "Samson", leeftijd: 35, isIngeschreven: true },  
];
```

js

Gegeven is onderstaande CSS in het bestand `combinatie.css`:

```
table {  
  border-collapse: collapse;  
  border: 1px solid black;  
}  
  
th,  
td {  
  border: 1px solid black;  
  padding: 0.5rem;  
}
```

CSS

```
text-align: left;
}
```

Gegeven is onderstaande HTML:

```
html
<!doctype html>
<html lang="nl">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Document</title>

    <link rel="stylesheet" href="./combinatie.css" />
    <script type="module" src="./combinatie.js"></script>
  </head>
  <body>
    <table>
      <thead></thead>
      <tbody></tbody>
    </table>
  </body>
</html>
```

Gebruik JavaScript om de tabelkop (`<thead>`) dynamisch te genereren op basis van de eigenschappen van de objecten in de array. Vervolgens moet je ook de rijen in de tabelbody (`<tbody>`) dynamisch genereren op basis van de gegevens in de array van objecten. Zorg ervoor dat als er een extra eigenschap aan de objecten wordt toegevoegd, deze ook automatisch in de tabel verschijnt.

Je mag er van uitgaan dat alle objecten in de array dezelfde eigenschappen hebben.

TIP

`Object.keys()` op het eerste element zal dezelfde eigenschappen teruggeven als `Object.keys()` op de andere elementen.

▼ Oplossing

html

```
<table>
  <thead></thead>
  <tbody></tbody>
</table>
```

CSS

```
table {
  border-collapse: collapse;
  border: 1px solid black;
}

th,
td {
  border: 1px solid black;
  padding: 0.5rem;
  text-align: left;
}
```

js

```
const studenten = [
  { naam: "John Duck", leeftijd: 34, isIngeschreven: true },
  { naam: "Jane Duck", leeftijd: 20, isIngeschreven: false },
  { naam: "Samson", leeftijd: 35, isIngeschreven: true },
];

const tableHead = document.querySelector("thead");
const tableBody = document.querySelector("tbody");

const headerRow = document.createElement("tr");
const eigenschappen = Object.keys(studenten[0]);
for (const eigenschap of eigenschappen) {
  const th = document.createElement("th");
  th.innerText = eigenschap;
  headerRow.appendChild(th);
}
```

```

tableHead.appendChild(headerRow);

for (const student of studenten) {
  const row = document.createElement("tr");
  for (const eigenschap of eigenschappen) {
    const td = document.createElement("td");
    td.innerText = student[eigenschap];
    row.appendChild(td);
  }
  tableBody.appendChild(row);
}

```

naam	leeftijd	isIngeschreven
John Duck	34	true
Jane Duck	20	false
Samson	35	true

Extra uitdagingen:

- Zorg dat de tekst in de tabelkop met een hoofdletter begint (bv. **Naam** in plaats van **naam**).
- Zorg dat i.p.v. **true** of **false** er **Ingeschreven** of **Niet ingeschreven** wordt weergegeven in de tabel.
- Filter alle studenten van 35 jaar of ouder uit de tabel (enkel studenten jonger dan 35 jaar worden weergegeven).

▼ Oplossing Extra uitdagingen

```

<table>
  <thead></thead>

```

html

```
<tbody></tbody>
</table>
```

CSS

```
table {
  border-collapse: collapse;
  border: 1px solid black;
}

th,
td {
  border: 1px solid black;
  padding: 0.5rem;
  text-align: left;
}
```

js

```
let studenten = [
  { naam: "John Duck", leeftijd: 34, isIngeschreven: true },
  { naam: "Jane Duck", leeftijd: 20, isIngeschreven: false },
  { naam: "Samson", leeftijd: 35, isIngeschreven: true },
];

studenten = studenten.filter((student) => student.leeftijd <
35); // Extra 3

const tableHead = document.querySelector("thead");
const tableBody = document.querySelector("tbody");

const headerRow = document.createElement("tr");
const eigenschappen = Object.keys(studenten[0]);
for (const eigenschap of eigenschappen) {
  const th = document.createElement("th");
  th.innerText = eigenschap.charAt(0).toUpperCase() +
eigenschap.slice(1); // Extra 1
  headerRow.appendChild(th);
}

tableHead.appendChild(headerRow);

for (const student of studenten) {
  const row = document.createElement("tr");
  for (const eigenschap of eigenschappen) {
```

```

const td = document.createElement("td");
// Extra 2
if (eigenschap === "isIngeschreven") {
  // Zelfde als student[eigenschap] === true
  if (student[eigenschap]) {
    td.innerText = "Ingeschreven";
  } else {
    td.innerText = "Niet ingeschreven";
  }
  // One-liner: td.innerText = student[eigenschap] ?
  "Ingeschreven" : "Niet ingeschreven";
} else {
  td.innerText = student[eigenschap];
}
row.appendChild(td);
}
tbody.appendChild(row);
}

```

Naam	Leeftijd	IsIngeschreven
John Duck	34	Ingeschreven
Jane Duck	20	Niet ingeschreven

Previous page

Theorie

Next page

Theorie

